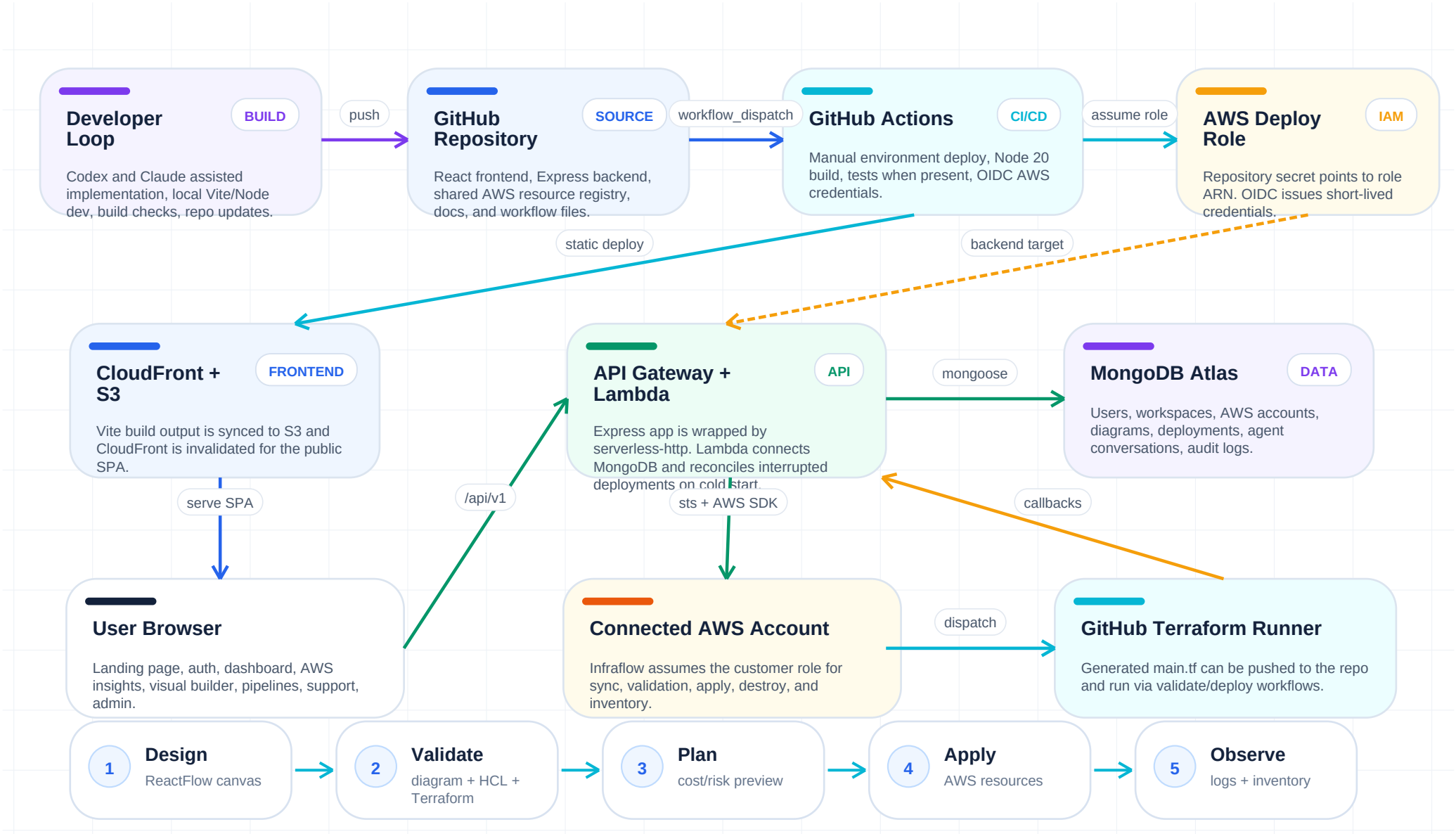
Infraflow Self-Deploying Architecture

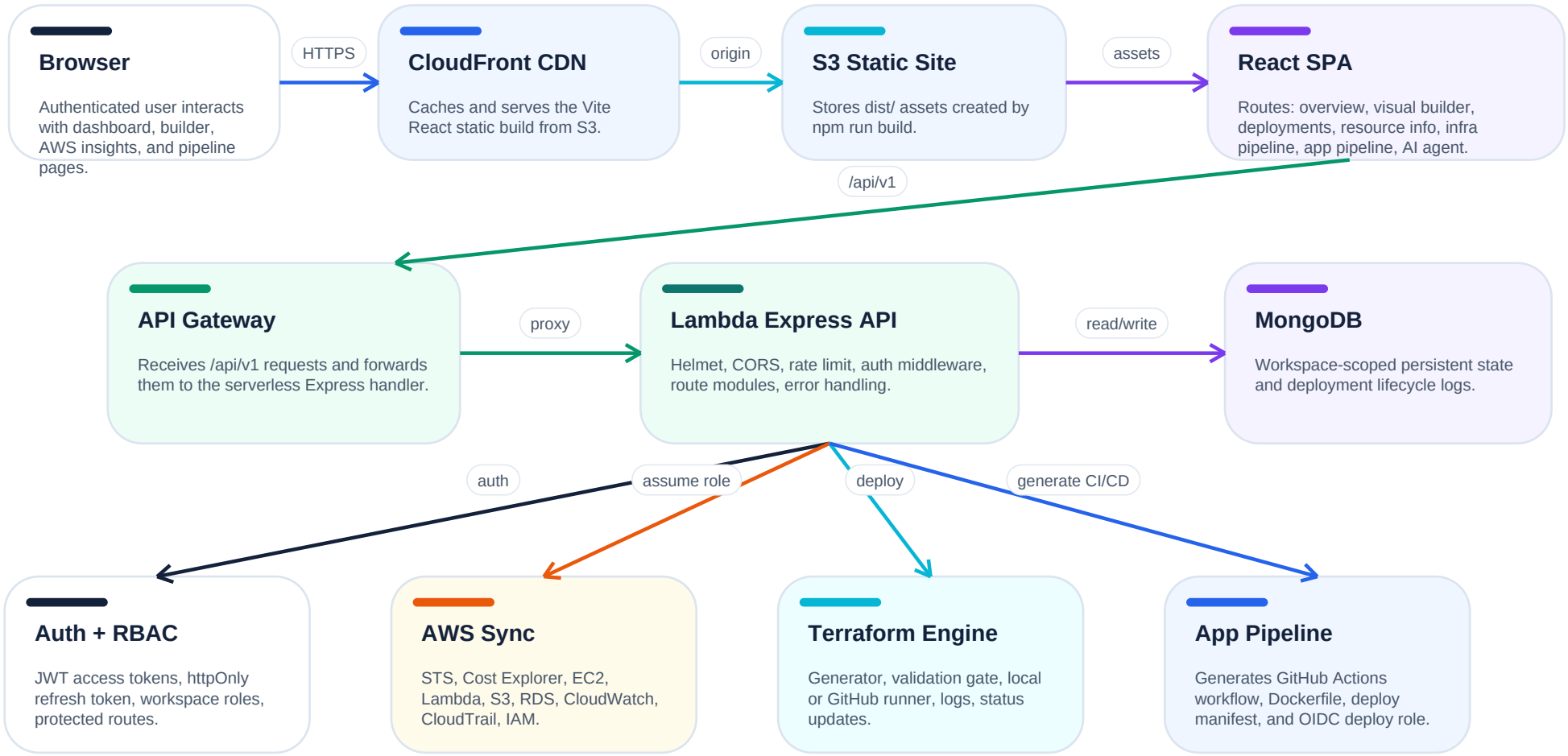
Interactive-style visual map of how this application is hosted, operated, and used to deploy infrastructure through its own pipeline.



Sources reviewed: README.md, docs/HLD.md, .github/workflows/infraflow-development-deploy.yml, .github/workflows/terraform-validate.yml ...

Runtime Request Flow

The deployed application separates the static SPA, serverless API, database state, and external cloud systems.

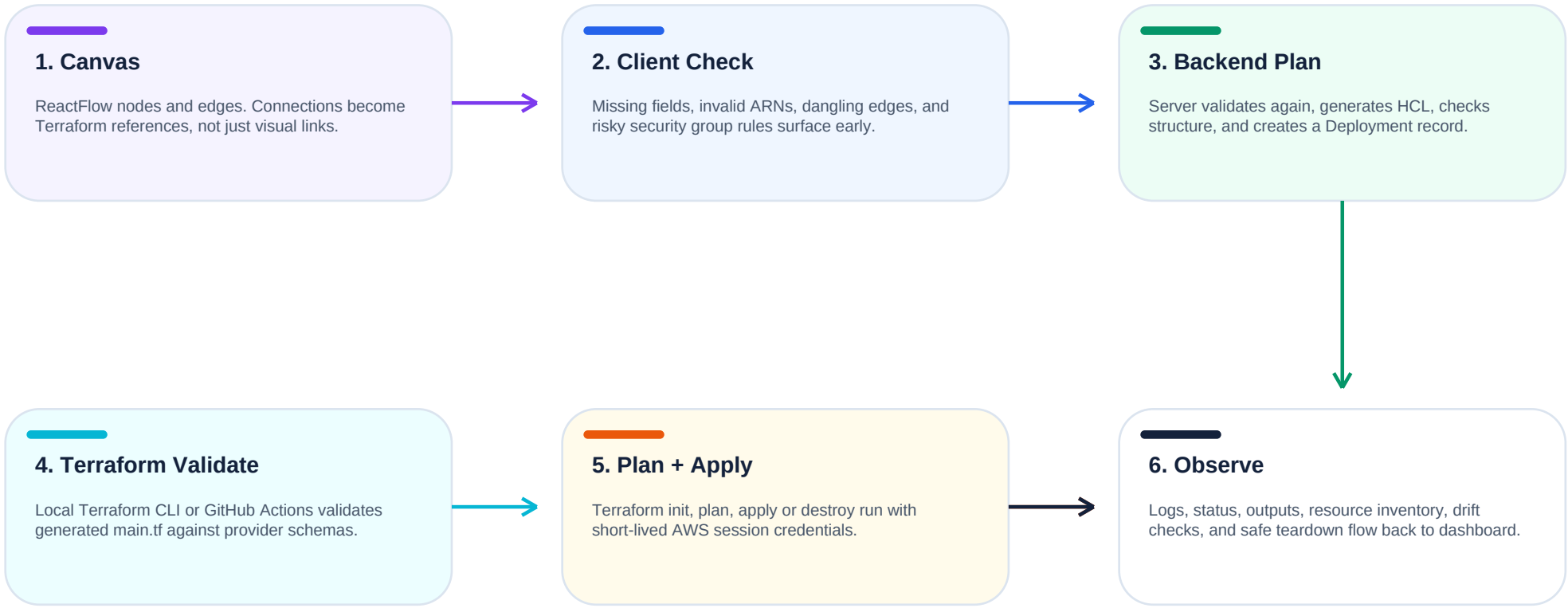


/auth /dashboard /diagrams /deployments /aws /agent /app-pipelines /github /notifications /tickets

Route surface extracted from IAAS backend/src/routes/index.js.

Infrastructure Deployment Flow

The visual builder turns a diagram into Terraform, then applies it only after repeated gates pass.



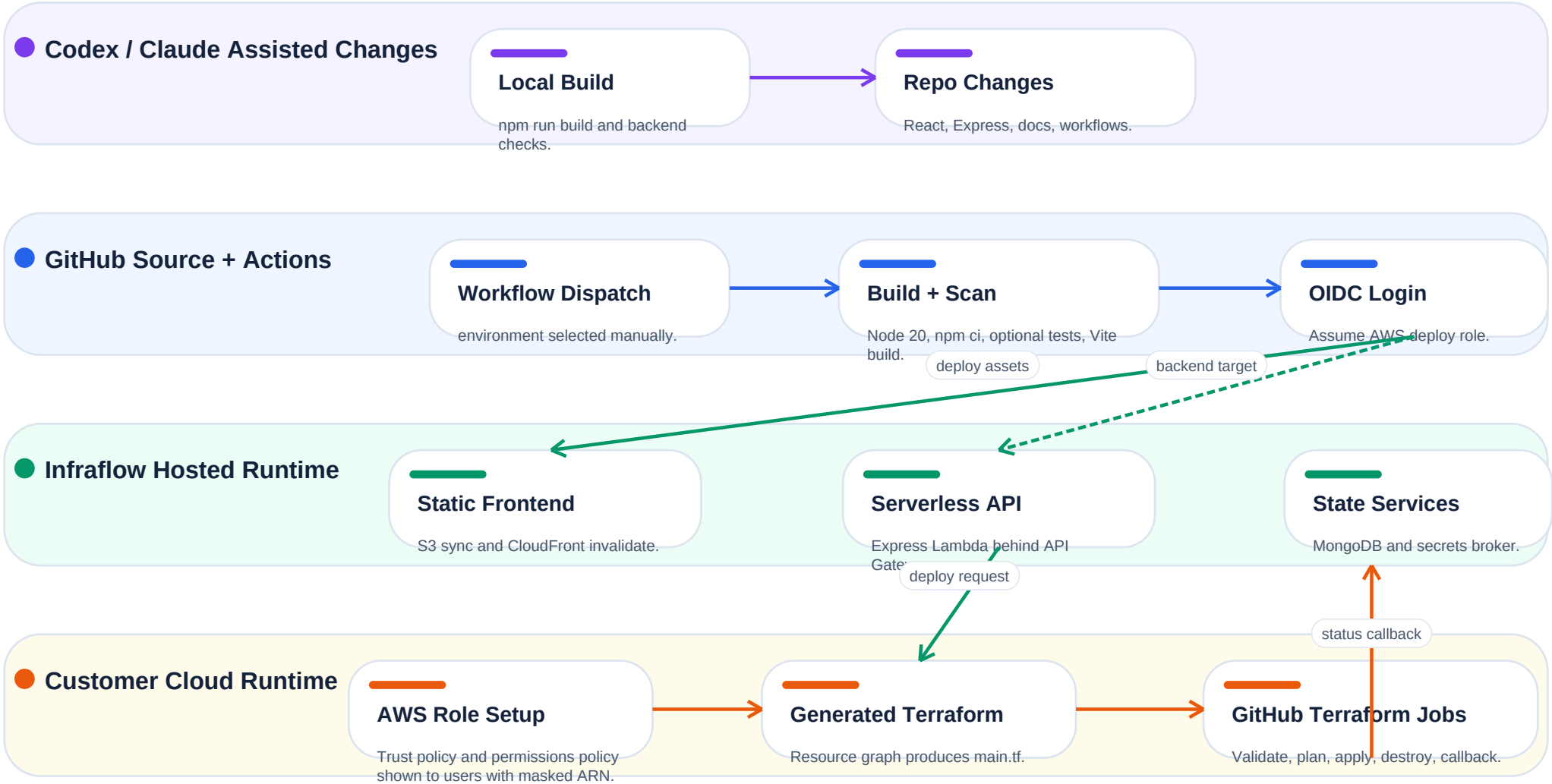
Deployment safety gates

- Workspace isolation
- Diagram validation
- Generated HCL check
- terraform validate
- terraform plan
- Auto cleanup on failed first apply

Executor options include local backend runner and GitHub-hosted Terraform workflows. Sensitive outputs can be moved behind Secrets Manager references.

Self-Pipeline and Ownership Model

How Infraflow deploys itself and how it provisions infrastructure for users without storing AWS keys.



Important current-state note

The repository includes Lambda entrypoint code and docs describing API Gateway + Lambda hosting. The inspected development deploy workflow clearly deploys the static frontend to S3/CloudFront and prepares AWS OIDC credentials; backend Lambda update should be verified before production release if not already handled elsewhere.

Sources reviewed: README.md, docs/HLD.md, .github/workflows/infraflow-development-deploy.yml, .github/workflows/terraform-validate.yml ...